

Quizify API Documentation

Base URL: `http://localhost:<PORT>`

Authentication

All protected routes require a Bearer token in the `Authorization` header:

```
Authorization: Bearer <token>
```

Tokens are issued by `POST /auth/login` and expire based on `JWT_EXPIRES_IN`.

Roles: `user` (default), `moderator`, `admin`. Some routes are restricted by role (noted per-endpoint).

Response envelope — every endpoint returns:

```
json
{
  "success": true | false,
  "message": "string",
  "data": { ... } | [ ... ]
}
```

Error responses — thrown errors are caught by a global error handler:

Code	Meaning
400	Validation error / bad input
401	Missing/invalid/expired token
403	Authenticated, but not allowed to access this resource
404	Resource not found
409	Conflict (duplicate entry)
500	Unexpected server error

Auth — /auth

POST /auth/register

Create a new account. **Public** (no auth required).

Body:

```
json

{
  "username": "string",
  "email": "user@example.com",
  "password": "min 8 characters"
}
```

Response 201:

```
json

{
  "success": true,
  "message": "Registered Successfully",
  "data": { "user_id": "uuid", "username": "...", "email": "...", "role": "user" }
}
```

POST /auth/login

Log in and receive a JWT. **Public**.

Body:

```
json

{
  "email": "user@example.com",
  "password": "string"
}
```

Response 200:

```
json
```

```
{
  "success": true,
  "message": "Logged in successfully!",
  "token": "jwt-string",
  "data": { "user_id": "uuid", "username": "...", "email": "...", "role": "..."}
}
```

Errors: `401` — wrong email or password.

Users — `/users`

`GET /users/myprofile/me`

Get the logged-in user's own profile. **Auth required.**

Response `200`: `data` = safe user fields (no password).

`PATCH /users/myprofile/me`

Update the logged-in user's own `username`/`email`. **Auth required.**

Body (all optional, at least one required):

```
json
{ "username": "string", "email": "string" }
```

`GET /users`

List all users. **Admin only.**

`PATCH /users/:id`

Update any user's `username`/`email`. **Admin only.**

Same body shape as `PATCH /users/myprofile/me`.

`DELETE /users/:id`

Delete a user. **Admin only.**

Response `200`: returns the deleted user's safe fields.

Categories — `/category`

`GET /category`

List all categories with a live `question_count`. **Auth required.**

Response `200`:

```
json
{
  "success": true,
  "data": [
    { "category_id": "uuid", "category_type": "geography", "user_id": "uuid", "
```



`GET /category/:id`

Get a single category (with `question_count`). **Auth required.**

Errors: `404` — category not found.

`POST /category`

Create a category. **Admin or moderator only.**

Body:

```
json
{ "category_type": "string" }
```

`PATCH /category/:id`

Rename a category. **Admin or moderator only.**

Body:

```
json
```

```
{ "category_type": "string" }
```

DELETE /category/:id

Delete a category. **Admin only.**

Questions — /question

GET /question

List all questions, with optional filters. **Auth required.**

Query params (all optional, combinable):

Param	Description
category	filter by exact category_type name
question_type	filter by exact question type
search	case-insensitive partial match on description

Example: GET /question?category=geography&question_type=singleChoice&search=capital

Response 200: data = array of questions, each including aggregated tags, images, and choices.

GET /question/:id

Get a single question by ID. **Auth required.**

Errors: 404 — question not found.

POST /question

Create a question with choices and tags/categories. **Admin or moderator only.**

Body:

```
json
```

```
{
  "description": "What is the capital of France?",
  "question_type": "singleChoice | multiChoice | openEnded",
  "tags": ["geography", "capitals"],
  "choices": [
    { "description": "Paris", "status": true },
    { "description": "London", "status": false }
  ]
}
```

Validation rules:

- `description`, `question_type`, `choices` are required (choices may be `[]` only for `openEnded`).
- `singleChoice`/`multiChoice` require ≥ 2 choices.
- `openEnded` must have **0** choices.
- Non-open-ended questions need ≥ 1 correct choice.
- `singleChoice` allows **exactly 1** correct choice.
- Each `tag` must match an existing `category_type`, or the request fails with `404`.

Response `201`: the created question, fully populated (tags/images/choices).

PATCH `/question/:id`

Partially update a question. **Admin or moderator only.**

Body (all optional, at least one required):

```
json
{
  "description": "string",
  "question_type": "singleChoice | multiChoice | openEnded",
  "tags": ["string"],
  "choices": [ { "description": "string", "status": true } ]
}
```

Sending `choices` replaces **all** existing choices. Sending `tags` replaces **all** existing category links.

Errors: `404` — question or category not found.

`DELETE /question/:id`

Delete a question (cascades to its choices, images, and category links). **Admin or moderator only.**

Contests — `/contest`

A contest moves through: `draft` → `saved` → `inProgress` → `completed`.

Ownership is not currently enforced. Any authenticated user can read/modify any contest by ID, regardless of who created it — this is a deliberate, temporary trade-off given the small expected user base (≤ 10 users). Revisit before opening this up to a larger or less-trusted group.

Admin

`GET /contest/admin`

List all contests, from every user. **Admin only.**

`GET /contest/admin/:id`

Get any single contest by ID. **Admin only.**

My Contests

`GET /contest/mycontests`

List the logged-in user's contests. **Auth required.**

`POST /contest/mycontests`

Create a new contest — currently starts as `saved` (not `draft`). A true `draft` state depends on a not-yet-built "target question count" feature; until that exists, every contest is created already `saved`. **Auth required.**

Body:

```
json
{ "contest_name": "string (must be unique)" }
```

`GET /contest/mycontests/:id`

Get a single contest. **Auth required.**

`PATCH /contest/mycontests/:id`

Rename a contest. **Auth required.**

Body:

```
json
{ "contest_name": "string" }
```

DELETE /contest/mycontests/:id

Delete a contest. **Auth required.**

Contest Lifecycle

PATCH /contest/mycontests/:id/start

Transition `draft`/`saved`/`completed` → `inProgress`. Restarting a `completed` contest is allowed by design — `completed` just indicates the contest has been played before, not that it's locked. **Auth required.**

Errors: `400` only if the contest is already `inProgress`.

PATCH /contest/mycontests/:id/finish

Transition `inProgress` → `completed`. **Auth required.**

Response `200`:

```
json
{
  "success": true,
  "message": "Contest has ended",
  "data": {
    "data": { "...contest row..." },
    "score": 7,
    "totalQuestions": 10
  }
}
```

Errors: `400` if the contest isn't currently `inProgress`.

Questions Within a Contest

GET /contest/mycontests/:id/questions

List all questions in a contest, ordered by `question_order`, including each question's per-contest `status` (`pending`/`correct`/`wrong`). **Auth required.**

POST `/contest/mycontests/:id/questions/random`

Replace the contest's question list with `Question_Count` randomly selected questions. **Auth required.**

Body:

```
json  
  
{ "Question_Count": 10 }
```

Errors: `400` if fewer than `Question_Count` questions exist in the bank.

GET `/contest/mycontests/:contest_id/questions/:question_id`

Get a single question within a specific contest (with its per-contest `status`/`order`). **Auth required.**

POST `/contest/mycontests/:contest_id/questions/:question_id`

Add one question to a contest. **Auth required.**

Body:

```
json  
  
{ "question_order": 1 }
```

Not yet handled: inserting a question already in the contest currently throws a raw duplicate-key error rather than a clean `409`.

PATCH `/contest/mycontests/:contest_id/questions/:question_id`

Update a question's order/position within a contest. **Auth required.**

Body:

```
json  
  
{ "question_order": 3 }
```

DELETE `/contest/mycontests/:contest_id/questions/:question_id`

Remove a question from a contest. **Auth required.**

PATCH `/contest/mycontests/:contest_id/questions/:question_id/answer`

Record the answer status for a question during an active contest. **Auth required.**

Body:

```
json
{ "status": "correct | wrong" }
```

Errors: 400 if the contest is not currently inProgress.

Data Model Reference

Table	Key Fields
users	user_id, username, email, password_hash, role
question	question_id, user_id, description, question_type
choice	question_id + choice_number (composite PK), status, description, image_path
category	category_id, category_type (unique), user_id, created_at
question_category	question_id + category_id (composite PK) — many-to-many
question_image	question_id, image_path
contest	contest_id, contest_name (unique), status, user_id, created_at, updated_at
question_contest	contest_id + question_id (composite PK), question_order, status

Question types: singleChoice, multiChoice, openEnded Contest statuses: draft, saved, inProgress, completed Question-in-contest statuses: pending, correct, wrong

Known Gaps (not yet implemented, deferred by project decision)

- Ownership enforcement on contests — any authenticated user can read/modify any contest by ID. Deferred given the small (≤10) expected user base; revisit if usage grows.
- Duplicate-question handling — adding a question already in a contest throws a raw DB error instead of a clean 409.
- Draft state for contests — createContest currently starts contests as saved rather than draft, pending a "target question count" feature.

- Image upload endpoint (paths are stored, but nothing handles file upload yet)
- History tracking ("previously used question" warnings)
- Admin stats dashboard
- Import/Export
- Bulk category reassignment
- Contest duplication
- Auto-replacement of deleted questions in existing contests
- Password hashing (currently stored in plaintext — known, deferred by project decision)